

Lista 2 - CIN7915-07342 (20251) - Data Science

João Pedro Becker Schneider

Inicialmente, é feito a importação dos pacotes necessários para construção do modelo de recomendação:

```
#importação das bibliotecas
import pandas as pd
import nltk
import re
import sklearn
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

#pacotes do NLTK
nltk.download('punkt')
nltk.download('punkt_tab') #tokenizador atual do nltk
nltk.download('stopwords') #dicionarios de stopwords
nltk.download('averaged_perceptron_tagger_eng') #modelo de POS tagging (Part-of-Speech).
```

Após, é preciso importar a fonte de dados utilizados para alimentar o modelo de recomendação:

```
#importando dataset
data = pd.read_csv("jazz.csv")

#data.head()
#data.sample()
```

Então, após importar tudo que se é necessário, o primeiro processamento ocorre transformando o texto bruto contido na tabela 'bio' para um texto processado e capaz de alimentar o modelo de recomendação.

```
#faz o pre-processamento da coluna 'bio'
def preprocess(text):
    if not isinstance(text, str):
        return ''
    text = text.lower() #converte texto para minusculo
    text = re.sub(r'[^w\s]', '', text) #apaga pontuacoes
    tokens = nltk.word_tokenize(text) #tokeniza o que sobra

    stopwords = set(nltk.corpus.stopwords.words('english')) #atribui stopwords
    tokens = [t for t in tokens if t not in stopwords and len(t) > 2] # remove stopwords e palavras pequenas

    tagged = nltk.pos_tag(tokens)
    tokens = [word for word, pos in tagged if pos.startswith('NN')] # filtra substantivos (tag filtering)

    stemmer = nltk.SnowballStemmer('english')
    tokens = [stemmer.stem(t) for t in tokens] #stemming

    return ' '.join(tokens)

#cria coluna nova com o que foi processado de bio
data['processed'] = data['bio'].apply(preprocess)

data[['name', 'processed']].head()
```

Em seguida, é criado o 'vetorizador' para os termos da tabela 'processed', resultante do pré-processamento.

```
#instancia vetorizador TF - IDF
vectorizer = TfidfVectorizer()

# Cada linha vira um vetor numérico, com pesos TF-IDF para cada termo
tfidf_matrix = vectorizer.fit_transform(data['processed'])

# (número de documentos, número de termos únicos)
print(f"Shape da matriz TF-IDF: {tfidf_matrix.shape}")
```

Por último, o modelo é alimentado pela matriz formada, e organizada do mais recomendado (com base na frequência dos termos) para o menos recomendado.

```
# Calcula a matriz de similaridade entre os artistas com base nos vetores TF-IDF
similarity_matrix = cosine_similarity(tfidf_matrix)

def recommend(index, top_n=5): #retorna os artistas mais similares ao artista no índice fornecido onde top_n é a quantidade de recomendações
    similarity_scores = list(enumerate(similarity_matrix[index]))
    similarity_scores = sorted(similarity_scores, key=lambda x: x[1], reverse=True)
    recommended_indices = [i for i, score in similarity_scores[1:top_n+1]]
    return data.iloc[recommended_indices][['name', 'bio']]

#teste
# print("Recomendações para:", data.loc[0, 'name'])
# print(recommend(0))
```